CHAPTER 4

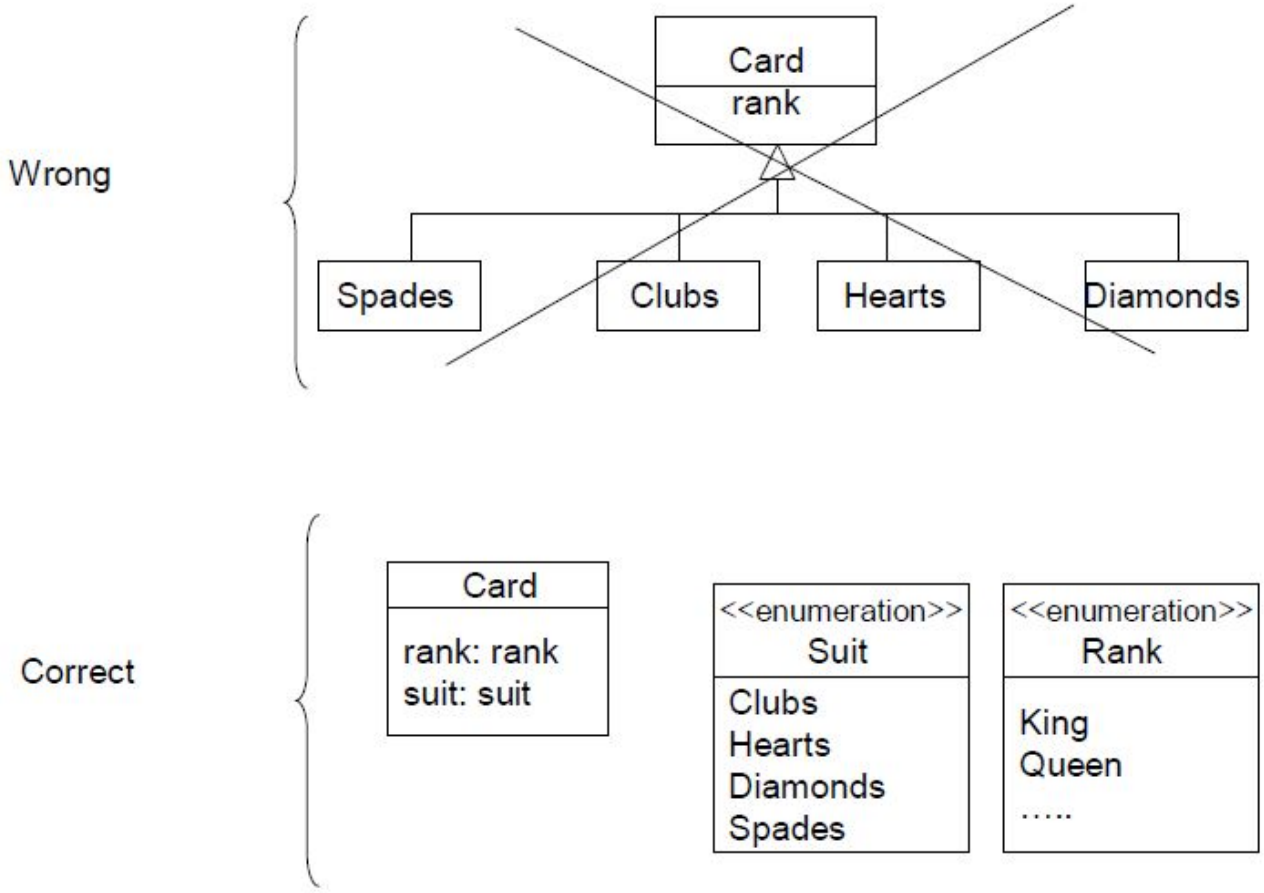
ADVANCED CLASS MODELING

Advanced Object and Class Concepts

- **Enumerations**
- A data type is a description of values, includes numbers, strings, enumerations.
- Enumerations: A Data type that has a finite set of values.
- Attribute *accessPermission* is an enumeration with possible values that include *read* and *read-write.*
- Eg: Boolean type= { TRUE, FALSE}
- Eg: figure.pentype _____ - - - - - --------
- Two diml.filltype 

- *Figure.PenType* is an enumeration that includes *solid,grey,dashed,* and *dotted.*
- *TwoDimensional.fillType is an enumeration that includes* solid,grey,none,horizontal lines, and vertical lines.
- When constructing a model, we should carefully note enumerations, because they often occur and are important to users.
- Do not use a generalization to capture the values of an Enumerated attribute.

- An Enumeration is merely a list of values; generalization is a means for structuring the description of objects.
- Introduce generalization only when at least one subclass has significant attributes, operations, or associations that do not apply to the superclass.
- In the UML an enumeration is a data type.
- We can declare an enumeration by listing the keyword *enumeration in guillemets* (<< >>) above the enumeration name in the top section of a box. The second section lists the enumeration values.

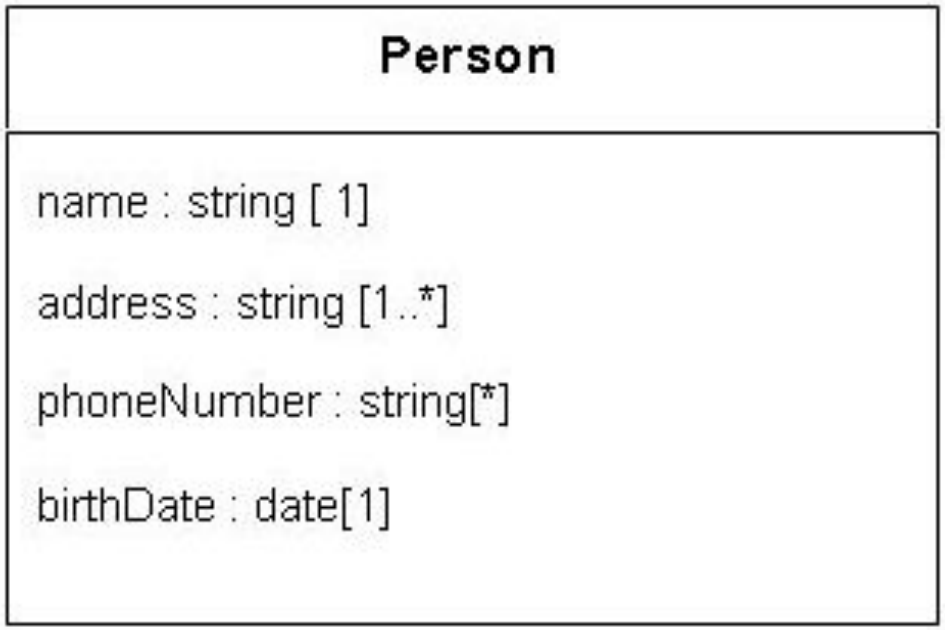


Modeling enumerations. Do not use a generalization to capture the values of an enumerated attribute

- Generalization should not be used for *Card*, because most games do not differentiate the behavior of spades,clubs,heatrs and diamonds.

Multiplicity

- Multiplicity is a collection on the cardinality of a set, also applied to attributes.
- Multiplicity of an attribute specifies the number of possible values for each instantiation of an attribute.
- The most common specifications are a mandatory single value [1] or an optional single [0...1] value and many[*].
- Multiplicity also indicates whether an attribute is single valued or can be a collection.

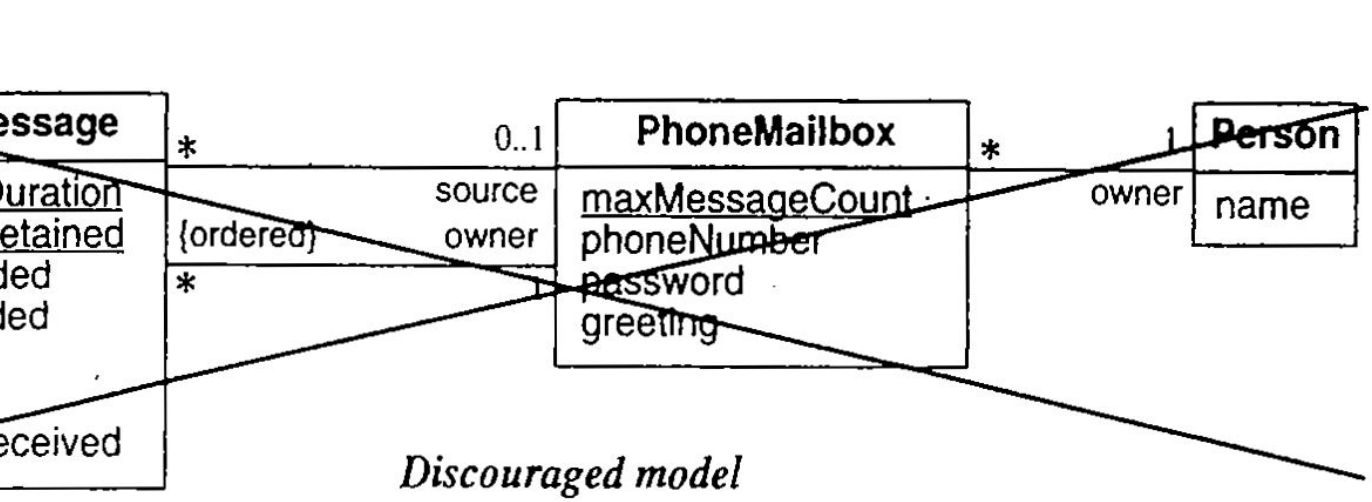


- A person has one name, one or more addresses, zero or more phone numbers, and one birthdate.

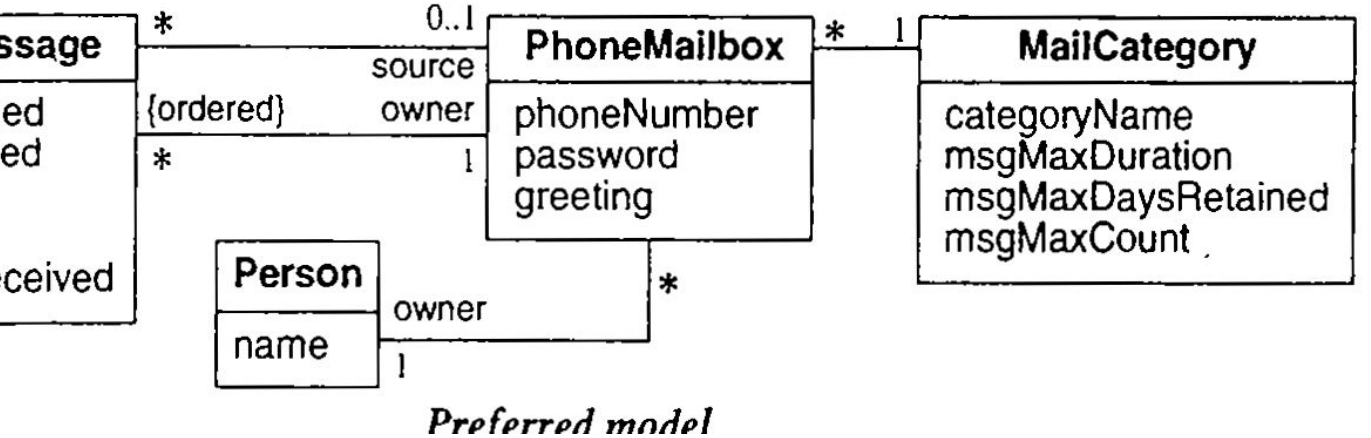
Scope

- Scope indicates if a feature applies to an object or a class.
- An underline distinguishes feature with class scope (static) from those with object scope.
- Our convention is to list attributes and operations with class scope at the top of the attribute and operation boxes, respectively.
- It is acceptable to use an attribute with class scope to hole the **extent of a class** - this is common with OO databases.

- Otherwise, you should avoid attributes with class scope because they can lead to an inferior model.
- It is better to model groups explicitly and assigns attributes to them.
- In contrast to attributes, it is acceptable to define operations of class scope.
- The most common use of class-scoped operations is to create new instances of a class, sometimes for summary data as well.



Discouraged model



Preferred model

- **Visibility**
- Visibility refers to the ability of a method to reference a feature from another class and has the possible values of *public, protected, private, and package.*
- Any method can access **public features.**
- Only methods of the containing class and its descendants via inheritance can access **protected features.**
- Only methods of the containing class can access **private features.**
- Methods of classes defined in the same package as the target class can access **package features**

- The UML denotes visibility with a prefix. “+” **public, “-” private, “#”protected, “~” package.**
- Lack of a prefix reveals no information about visibility.
- Several issues to consider when choosing visibility are
- **Comprehension:** understand all public features to understand the capabilities of a class. In contrast we can ignore private, protected, package features – they are merely an implementation convince.

- **Extensibility:** many classes can depend on public methods, so it can be highly disruptive to change their signature.
- Since fewer classes depend on private, protected, and package methods, there is more latitude to change them.
- **Context:** private, protected, and package methods may rely on preconditions or state information created by other methods in the class.
- Applied out of context, a private method may calculate incorrect results or cause the object to fail.

Associations ends

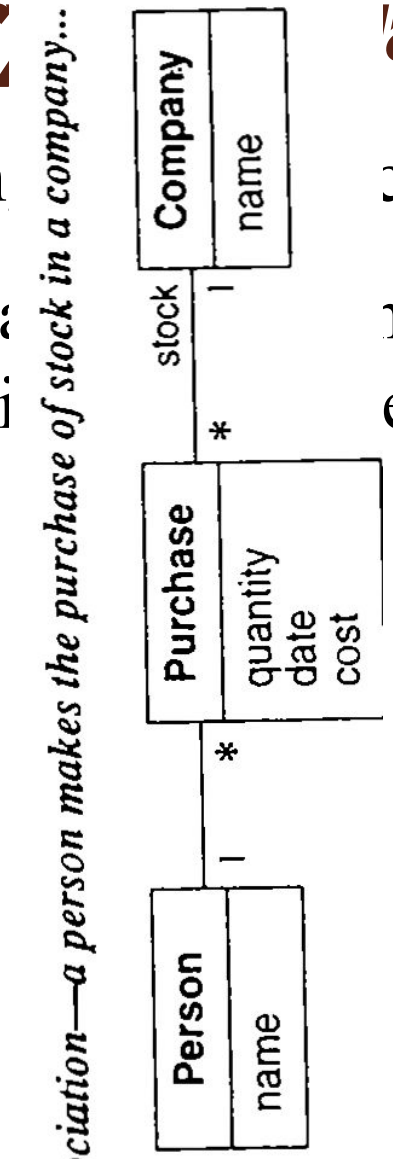
- Association End is an end of association.
- A binary association has 2 ends; a ternary association has 3 ends.
- Association Properties
  - Association end name
  - Multiplicity
  - Ordering
  - Bags and Sequences
  - Qualification
  - Aggregation: The association end may be an aggregate or constituent part. Only a binary association can be aggregation; one association end must be an aggregate and the other must be a constituent.

- Changeability: This property specifies the update status of an association end. The possibilities are changeable and readonly.
- Navigability: An association may be traversed in either direction.

- UML shows navigability with an arrowhead on the association end attached to the target class.
- Arrowheads may be attached to zero,one, or both ends of an association.
- Visibility : Similar to attributes and operations, association ends may be *public, protected, private, or package.*

N-ar[illegible]ation

- Association amon[illegible]classes.
- Most of them ca[illegible]mposed into binary associations, wi[illegible]e qualifiers and attributes.



...n-ary association—a person makes the purchase of stock in a company...

...ed as...

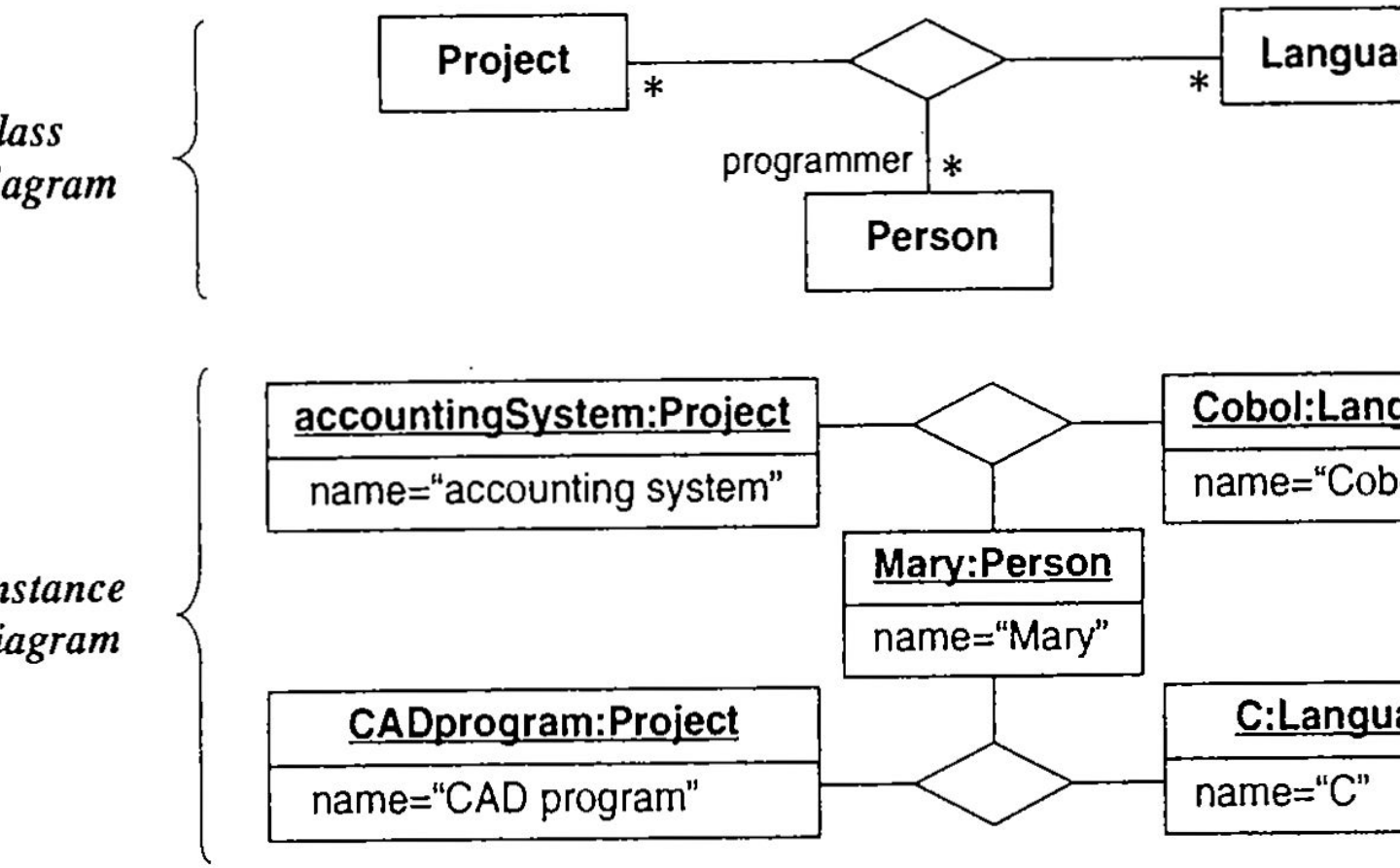


Figure 4.6 Ternary association and links. An n-ary association can hav
association end names, just like a binary association.

- The UML symbol for n-ary associations is a diamond with lines connecting to related classes. If the association has a name, it is written in italics next to the diamond.
- The OCL does not define notation for traversing n-ary associations.
- A typical programming language cannot express n-ary associations. So, promote n-ary associations to classes.
- Be aware that you change the meaning of a model, when you promote n-ary associations to classes.
- An n-ary association enforces that there is at most one link for each combination.

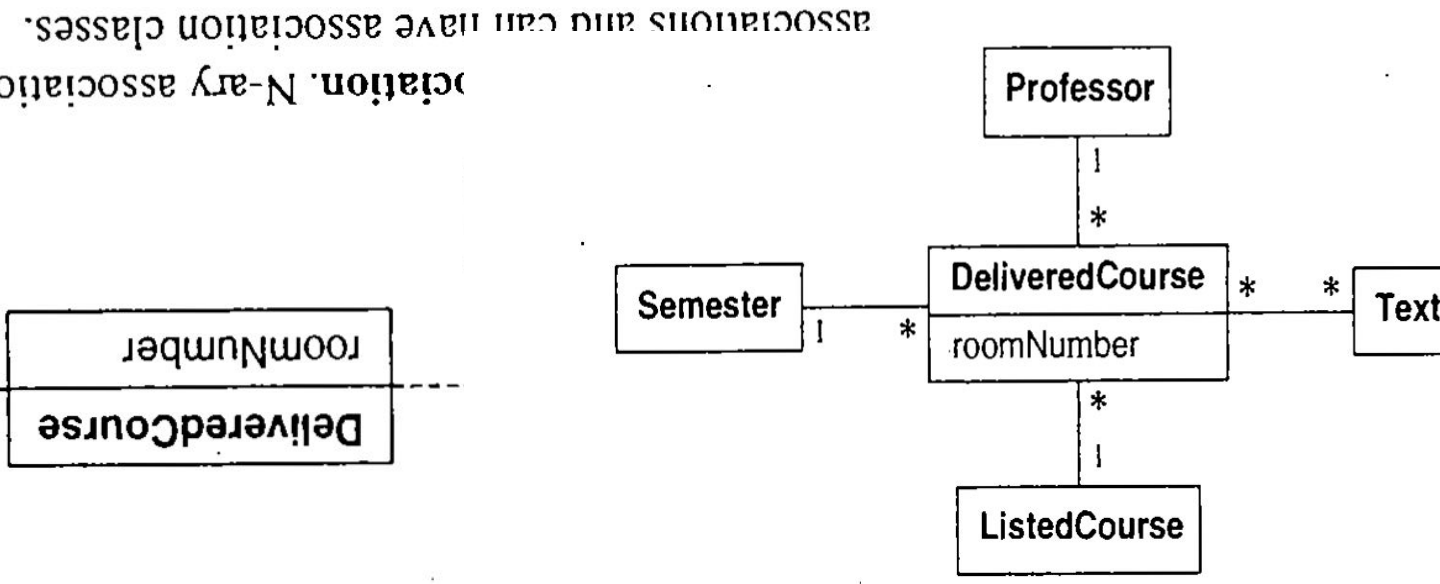


Figure 4.8 Promoting an n-ary association. Programming lan
express n-ary associations, so you must promote the

- Ex: Professor teaches a listed course during a semester. The resulting delivered course has a room number and any number of textbooks.
- An n-ary association enforces that there is at most one link for each combination- for each combination of *Professor, Semester* and *ListedCourse* there is one *DeliveredCourse.*
- Promoted class permits any number of links- for each combination of *Professor, Semester* and *ListedCourse* there can be many *DeliveredCourse.*
- Special application code would have to enforce the uniqueness of *Professor +Semester +ListedCourse.*

Aggregation

- Aggregation is a strong form of association in which an aggregate object is made of constituent parts.
- Constituents are the *part* of aggregate.
- The aggregate is semantically an extended object that is treated as a unit in many operations, although physically it is made of several lesser objects.
- We define an aggregation as relating an assembly class to one constituent part class.
- An assembly with many kinds of constituent parts corresponds to many aggregations.

e 4.9 Aggregation. Aggregation is a kind of association in which an aggregate object is made of constituent parts.

- Ex: a *LandMower* consists of a *Blade*, an *Engine*, many *Wheels* and a *Deck*. *LawnMower* is the assembly and the other parts are constituents.
- *LawnMower* to *Blade* is one aggregation, *LawnMower* to *Engine* is another aggregation.

- We define each individual pairing as an aggregation so that we can specify the multiplicity of each constituent part within the assembly.
- This definition emphasizes that aggregation is a special form of binary association.
- The most significant property of aggregation is transitivity (if A is part of B and B is part of C, then A is part of C) and antisymmetric (if A is part of B then B is not part of A).

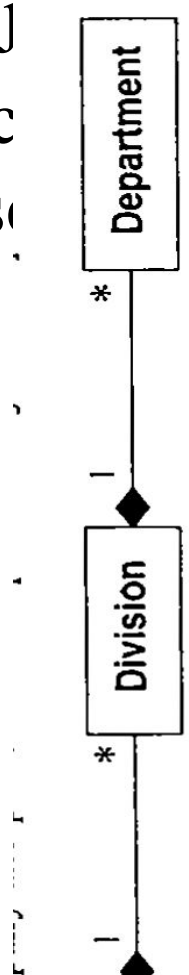
Aggregation versus Association

- Aggregation is a special form of association, not an independent concept.
- Aggregation adds semantic connotations.
- If two objects are tightly bound by a part-whole relationship, it is an aggregation.
- If the two objects are usually considered as independent, even though they may often be linked, it is an association.
- Aggregation is drawn like association, except a small (hollow) diamond indicates the assembly end.

Aggregation versus Composition

- The UML has 2 forms of part-whole relationships: a general form called Aggregation and a more restrictive form called composition.
- Composition is a form of aggregation with two additional constraints.
- A constitute part can belong to at most one assembly.
- Once a constitute part has been assigned an assembly, it has a coincident lifetime with the assembly.

- Composition implies ownership of the parts by the whole.
- This can be convenient for programming: Deletion of an assembly object triggers deletion of all constituent obj ı.
- Notation for c ll solid diamond next to the ass



mposition. With composition a constituent part belongs to at
st one assembly and has a coincident lifetime with the assembly.

For

Propagation of Operations

- Propagation (triggering) is the automatic application of an operation to a network of objects when the operation is applied to some starting object.
- For example, moving an aggregate moves its parts; the move operation propagates to the parts.
- Provides concise and powerful way of specifying a continuum behavior.
- Propagation is possible for other operations including save/restore, destroy, print, lock, display.

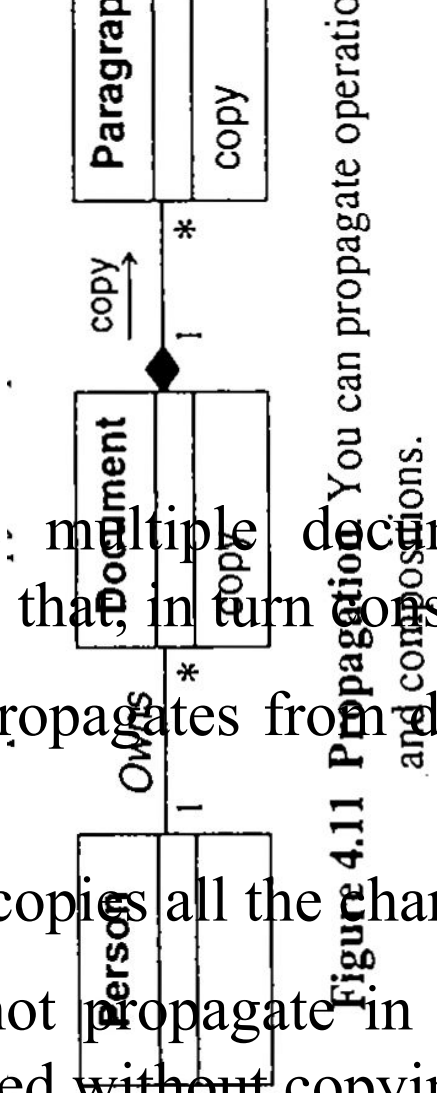


Figure 4.11 Propagation. You can propagate operations along aggregations and compositions.

- Ex: A person owns multiple documents. Each document consists of paragraphs that, in turn consist of characters.
- The copy operation propagates from documents to paragraphs to characters.
- Copying a paragraph copies all the characters in it.
- The operation does not propagate in the reverse direction; a paragraph can be copied without copying the whole document.
- Similarly, copying a document copies the owner link but does not spawn a copy of the person who is owner.

- Deep Copy: copy an entire network
- Shallow copy: copy the starting object and none of the related objects.
- Propagation on class models with a small arrow indicating the direction and operation name next to the affected association.
- The notation binds propagation behavior to an association, direction, and operation.

Abstract Classes

- Abstract class is a class that has no direct instances but whose descendant classes have direct instances.
- A concrete class is a class that is instantiable; that is, it can have direct instances.
- A concrete class may have abstract class.
- Only concrete classes may be leaf classes in an inheritance tree.

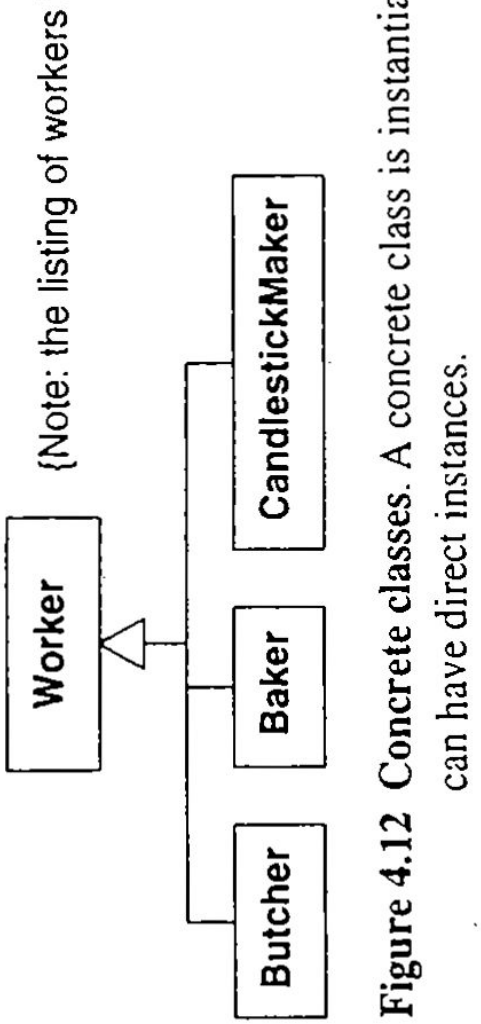


Figure 4.12 Concrete classes. A concrete class is instantia
can have direct instances.

- *Butcher, Bake* *stickMaker* are concrete class ey have direct instances.
- *Worker* also is a concrete class because some occupations may not be specified.

Abstract class and abstract operation. An abstract class is a class that has no direct instances

- In UML class name is listed in ord {abstract} below or after the name).

- We can use abstract classes to define the methods that can be inherited by subclasses.
- Alternatively, an abstract class can define the signature for an operation with out supplying a corresponding method. We call this an ***abstract operation.***
- Abstract operation defines the signature of an operation for which each concrete subclass must provide its own implementation.
- A concrete class may not contain abstract operations, because objects of the concrete class would have undefined operations.

- An abstract operation is designated by italics or the keyword {abstract}.
- *ComputePay* is an abstract operation of class Employee; its signature but not its implementation is defined.
- Each subclass must supply a method for this operation.
- It is a good idea to avoid concrete superclasses.
- All superclasses are abstract and all leaf classes are concrete.

- It is awkward where a concrete superclass must both specify the signature of an operation for descendent classes and also provide an implementation for its concrete instances.
- We can eliminate concrete superclasses by introducing an *Other* subclass.

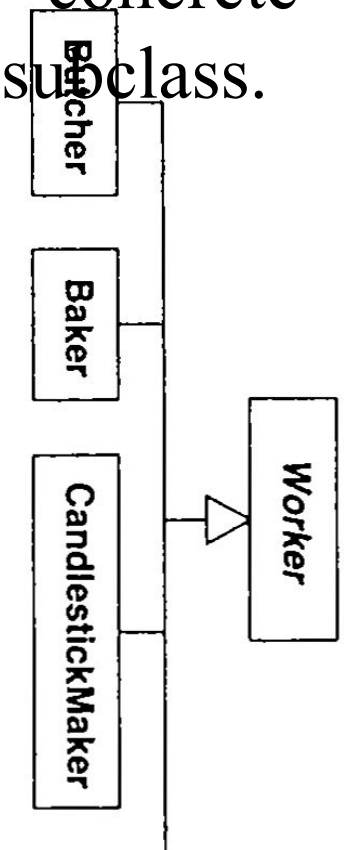


Figure 4.4 Avoiding concrete superclasses. You can a
concrete superclasses by introducing an *Oth*

Multiple Inheritance

- Multiple inheritance permits a class to have more than one superclass and to inherit features from all parents.
- We can mix information from 2 or more sources.
- This is a more complicated from of generalization than single inheritance, which restricts the class hierarchy to a tree.
- The advantage of multiple inheritance is greater power in specifying classes and an increased opportunity for reuse.

- The disadvantage is a loss of conceptual and implementation simplicity.
- The term multiple inheritance is used somewhat imprecisely to mean either the conceptual relationship between classes or the language mechanism that implements that relationship.

Kinds of Multiple Inheritance

- The most common form of multiple inheritance is from sets of disjoint classes.
- Each subclass inherits from one class in each set.
- The appropriate combinations depend on the needs of an application.
- Each generalization should cover a single aspect.
- We should use multiple generalizations if a class can be refined on several distinct and independent aspects.

Multiple inheritance from disjoint classes. This is the most common form of multiple inheritance.

- *FullTimeIndi* is both *FullTimeEm* *ontributor* and combines the
- *FullTimeEm* *Employee* are disjoint; each employee must belong to exactly one of these.

- Similarly, *Manager* and *IndividualContributor* are also disjoint and each employee must be one or the other.
- We could define three additional combinations: *FullTimeManager, PartTimeIndividualContractor, and PartTimeManager.*
- A subclass inherits a feature from the same ancestor class found along more than one path only once; it is the same feature.
- *FullTimeIndividualContributor* inherits *Employee* features along two paths, via *employeeStatus* and *managerialStatus.*
- Each *FullTimeIndividualContributor* has only a single copy of *Employee* features.

- Conflicts among parallel definitions create ambiguities that implementations must resolve.
- Ex:*FullTimeEmployee* and *IndividualContributor* both have an attribute called *name*. *FullTimeEmployee.name* could refer to the person's full name while *IndividualContributor .name* refers to the person's title.
- The best solution is to restate the attributes as *FullTimeEmployeee.personName* and *IndividualContributor.title*.

- Multiple Inheritance can also occur with overlapping classes.
- Ex: *AmphibiousVehicle* is both *LandVehicle* and *WaterVehicle*.
- *LandVehicle* and *WaterVehicle* overlap, because some vehicles travel on both land and water.
- The UML uses a constraint to indicate an overlapping generalization set; the notation is a dotted line cutting across the affected generalization with keywords in braces.

- *Overlapping* means that an individual vehicle may belo[illegible] subclasses.
- *Incomple* subclasses of vehicle h[illegible]ned.

16 **Multiple inheritance from overlapping classes.** This form of multiple inheritance occurs less often than with disjoint classes.

Multiple Classification

- An instance of a class is inherently an instance of all ancestors of the class.
- For example, an instructor could be both faculty and student.
- But what about a Harvard Professor taking classes at MIT? There is no class to describe the combination. This is an example of multiple classification, in which one instance happens to participate in two overlapping classes.

Figure 4.17 Workaround for multiple classification. OO lan
handle this well, so you must use a workaround.

- UML Permits Multiple classification but most OO languages handle it poorly.
- The best approach using conventional languages is to treat Person as an object composed of multiple *UniversityMember* objects.
- The workaround replaces inheritance with a delegation.

Workarounds

- Dealing with lack of multiple inheritance is really an implementation issue, but early restructuring of a model is often the easiest way to work around its absence.
- Here we list 2 approaches for restructuring techniques (it uses delegation)
- Delegation is an implementation mechanism by which an object forwards an operation to another object for execution.

- **Delegation using composition of parts:**
- Here we can recast a superclass with multiple independent generalization as a composition in which each constituent part replaces a generalization.
- This is similar to multiple classification. This approach replaces a single object having a unique ID by a group of related objects that compose an extended object. Inheritance of operations across the composition is not automatic.
- The composite must catch operations and delegate them to the appropriate part.

- *EmployeeEmp* … superclass of *FullTimeEmpl* … *ployee.*
- Then you ca … s a composition of *EmployeeE* … *yeeManagement.*
- An operation … bject would have to be redirected to *EmployeeEmployment* or *EmployeeManagement* part by the *Employee* class.

- **Inherit the most important class and delegate the re**

- Figure preserve inheritance across the most import n.
- We degrade t generalization to composition an operations as in previous alterna

Figure 4.19 Workaround for multiple inheritance—inheri

- **Nested generalization:** this approach multiplies out all possible combinations
- This preserv ates declarations and code an programming.
- *FullTimeEm ployee,* add two subclasses f ial contributors.

Figure 4.20 Workaround for multiple inheritance—nested gene

Issues to consider when selecting the best workaround

- **Superclasses of equal importance:** if a subclass has several superclasses, all of equal importance, it may be best to use delegation and preserve symmetry in the model.
- **Dominant superclass:** if one superclass clearly dominates and the others are less important, preserve inheritance through this path.
- **Few subclasses**: if the number of combinations is small, consider nested generalization. If the number of combinations is large, avoid it.

- **Sequencing generalization sets:** if we use generalization, factor on the most important criterion first, the next most important second, and so forth.
- **Large quantities of code:** try to avoid nested generalization if we must duplicate large quantities of code.
- **Identity:** consider the importance of maintaining strict identity. Only nested generalization preserves this.

Metadata

- Metadata is data that describes other data. For example, a class definition is a metadata.
- Models are inherently metadata, since they describe the things being modeled.
- Many real-world applications have metadata, such as parts catalogs, blueprints, and dictionaries. Computer-languages implementations also use metadata heavily.

mple of metadata. Metadata often arises in applications.

- Ex: A car mod best price, and a manufacturer.
  - 1969 F
  - 1975 Vo
- A physical Car has a serial number,color,options, and an owner.
  - John Doe may own a blue Ford with serial number IFABP

- a red Volkswagen with serial number 7E81F.
- A car model describes many physical cars and holds common data. A car model is metadata relative to a physical car, which is a data.
- We can also consider classes as objects, but classes are meta-objects and not real world objects.
- Class descriptor object have features, and they in turn have their own classes, which are called ***metaclasses.***

- ***Reification***

- Reification is the promotion of something that is not an object into an object.

- Reification is a helpful technique for Meta applications because it lets you shift the level of abstraction.

- On occasion it is useful to promote attributes, methods, constraints, and control information into objects so you can describe and manipulate them as data.

- As an example of reification, consider a database manager.
- A developer could write code for each application so that it can read and write from files.
- Instead, for many applications, it is better idea to reify the notion of data services and use a database manager.
- A database manager has abstract functionality that provides a general- purpose solution to accessing data reliably and quickly for multiple users.

Figure 4.22 Reification. Reification is the promotion of some
an object into an object and can be helpful for

- Ex: pron ttribute to a class to captur ationship between *Substanc*
- A chemical substance may have multiple aliases.

  For Ex, propylene may be referred to as propylene and C3H6.

Constraints

- Constraint is a condition involving model elements, such as objects, classes, attributes, links, associations, and generalization sets.
- A Constraint restricts the values that elements can assume by using OCL.
- **Constraints on objects**
- No employee's salary can exceed the salary of the employee's boss.(constraint between two things at the same time)
- No window can have the aspect ratio (length/width) of less than 0.8 or greater than 1.5(constraint between attributes o a single object)

- The priority of a ...crease(constraint on the same obj...

Figure 4.23 Constraints on objects. The structure of a model expresses ... constraints, but sometimes it is helpful to add explicit const...

- **Constraints o... ...on sets**
- Class models c... ...onstraints through their very struct...
- For example, ... of generalization imply certain st... ...nts.

- With single inheritance the subclasses are mutually exclusive. Furthermore, each instance of an abstract superclass corresponds to exactly one subclass instance.
- Each instance of a concrete superclass corresponds to at most one subclass instance.
- The UML defines the following keyword s for generalization.
- **Disjoint:** The subclasses are mutually exclusive. Each object belongs to exactly one of the subclasses.

- **Overlapping:** The subclasses can share some objects. An object may belong to more than one subclass.
- **Complete:** The generalization lists all the possible subclasses**.**
- **Incomplete:** The generalization may be missing some subclasses**.**
- **Constraints on Links**
- Multiplicity is a constraint on the cardinality of a set. Multiplicity for an association restricts the number of objects related to a given object.

- Multiplicity for an attribute specifies the number of values that are possible for each instantiation of an attribute.
- Qualification also constraints an association. A qualifier attribute does not merely describe the links of an association but is also significant in resolving the “many” objects at an association end.
- An association class implies a constraint. An association class is a class in every right;
- For example, it can have attribute and operations, participate in associations, and participate in generalization.

- But an association class has a constraint that an ordinary class does not; it derives identity from instances of the related classes.
- An ordinary association presumes no particular order on the object of a “many” end.
- The constr indicates that the elements of ociation end have an explicit orde reserved.

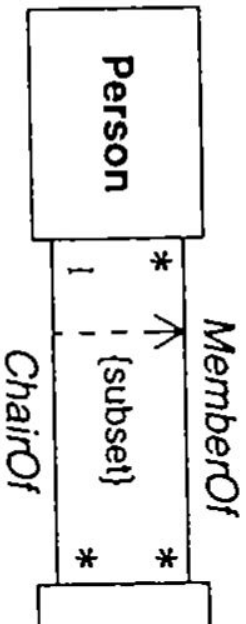


Figure 4.24 Subset constraint betwe

Use of constraints

- It is good to express constraints in a declarative manner. Declaration lets you express a constraint's intent, without supposing an implementation.
- Typically, we need to convert constraints to procedural form before we can implement them in a programming language, but this conversion is usually straightforward.

- A “good” class model captures many constraints through its structure. It often requires several iterations to get the structure of a model right from the prospective of constraints. Enforce only the important constraints.
- The UML has two alternative notations for constraints; either delimit a constraint with braces or place it in a “dog-earned” box. We can use dashed lines to connect constrained elements.
- A dashed arrow can connect a constrained element to the element on which it depends.

Derived Data

- A derived element is a function of one or more elements, which in turn may be derived.
- A derived element is redundant, because the other elements completely determine it.
- Ultimately, the derivation tree terminates with base elements. Classes, associations, and attributes may be derived.
- The notation for a derived element is a slash in front of the element name along with constraint that determines the derivation.

- Ex: A machine c veral assemblies that in turn consist n assembly has a geometrical off spect to machine coordinates.
- Each part has an respect to assembly coordinates.
- We can define a coordinate system for each part that is derived from machine coordinates, assembly offset, and part offset.

- The coordi
derived clas
a derived as

presented as a
to each part by
set.

Figure 4.26 Derived object and association. Derived data can com
plementation, so only use derived data where it truly is

Packages

- A ***package*** is a group of elements (classes, association, generalization, and lesser packages) with a common theme.
- A package partitions a model, making it easier to understand and
- Packages form a
the root, which is

kage partitions a model
nd and manage. Large
ers of packages.
asing abstraction toward
the top-level package.

n for a package. Packages let you organize large
so that persons can more readily understand them.

- There are various themes for forming packages: dominant classes, dominant relationships, major aspects of functionality, and symmetry.
- We could divide the class model of a compiler into packages for lexical analysis, parsing, semantic analysis, code generation, optimization.
- We can offer the following tips for devising packages.
- **Carefully delineate each package's scope**:
- The precise boundaries of a package are a matter of judgment. Like other aspects of modeling, defining the scope of a package requires planning and organization.

- Make sure that class and association names are unique within each package, and use consistent names across packages as much as possible.
- **Define each class in a single package**:
- The defining package should show the class name, attributes, and operations.
- Other packages that refer to a class can use a class icon, a box that contains only the class name.
- **Make packages cohesive**:
- Associations and generalizations should normally appear in a single package, but classes can appear in multiple packages.